

ЛАБОРАТОРНА РОБОТА № 4

Тема: Автоматизація (CI/CD).

Мета: Отримати практичний досвід роботи щодо автоматизації рутинних задач.

Час виконання: 2 год.

ХІД РОБОТИ

Теоретичні відомості

Безперервна інтеграція (Continuous Integration, CI) і безперервне постачання (Continuous Delivery, CD) представляє собою культуру, набір базових принципів і практик, які дозволяють розробникам частіше і надійніше розгортати зміни в програмному забезпеченні.

CI/CD – це одна з DevOps-практик. Вона також відноситься і до agile-практики: автоматизація розгортання дозволяє розробникам зосередитися на реалізації бізнес-вимог, якості коду та безпеки.

Визначення CI/CD

Безперервна інтеграція — це методологія розробки та набір практик, згідно яких в код вноситься невеликі зміни з частими комітами. І оскільки більшість сучасних додатків розробляються з використанням різних платформ та інструментів, то виникає потреба в механізмі інтеграції та тестуванні змін, що вносяться.

З технічної точки зору, мета CI — забезпечити послідовний та автоматизований спосіб збирання та тестування додатків. При налагодженому процесі безперервної інтеграції розробники з більшою ймовірністю робитимуть часті коміти, що, у свою чергу, сприятиме покращенню комунікації та підвищенню якості програмного забезпечення.

Безперервне постачання починається там, де закінчується безперервна інтеграція. Вона автоматизує розгортання додатків у різні оточення: більшість розробників працюють як із продакшн-оточенням, так і з середовищами розробки та тестування.

Інструменти CI/CD допомагають налаштовувати специфічні параметри оточення, які конфігуруються під час розгортання. А також CI/CD-автоматизація виконує необхідні запити до веб-серверів, баз даних та інших сервісів, які можуть вимагати перезапуску або виконання якихось додаткових дій під час розгортання програми.

Безперервна інтеграція та безперервне постачання вимагають безперервного тестування, оскільки кінцева мета — розробка якісних додатків. Безперервне тестування часто реалізується у вигляді набору різних автоматизованих тестів (регресійних, продуктивності та інших), які виконуються у CI/CD-конвеєрі.

CI/CD дозволяє реалізувати безперервне розгортання: при успішному проходженні коду через CI/CD-конвеєр, збірки автоматично розгортаються у продакшн-оточенні. Команди, які практикують безперервне постачання, можуть дозволити собі щоденне або навіть щогодинне розгортання. Хоча тут варто відзначити, що безперервне постачання підходить не для всіх бізнес-додатків.

Безперервна інтеграція покращує комунікацію та якість

Безперервна інтеграція - це методологія розробки, заснована на регламентованих процесах та автоматизації. При впровадженій безперервній інтеграції розробники часто комітують свій код в репозиторій вихідного коду. І більшість команд дотримуються правил - комітити як мінімум один раз на день. При невеликих змінах простіше виявляти дефекти та різні проблеми, ніж при великих змінах, над якими працювали протягом тривалого часу. Крім того, робота з короткими циклами комітів зменшує ймовірність зміни однієї і тієї ж частини коду кількома розробниками, що може призвести до конфліктів при злитті.

Команди, що впроваджують безперервну інтеграцію, часто починають із налаштування системи контролю версій та визначення порядку роботи. Незважаючи на те, що коміти робляться часто, реалізація фіч і виправлення багів можуть виконуватися досить довго. Для контролю того, які фічі та код готові існує кілька підходів.

Багато хто використовують фіча-прапори (feature flag) - механізм для включення та вимкнення функціоналу в рантаймі. Функціонал, який ще знаходиться на стадії розробки, обертається фіча-прапорами і розгортається з master-гілки в продакшн, але відключається доти, доки не буде повністю готовий до використання. За даними недавнього дослідження 63 відсотки команд, які використовують фіча-прапори, говорять про покращення тестованості та підвищення якості програмного забезпечення. Для роботи з фіча-прапорами є спеціальні інструменти, такі як CloudBees Rollout, Optimizely Rollouts та LaunchDarkly, які інтегруються в CI/CD та дозволяють виконувати конфігурацію на рівні фіч.

Інший спосіб роботи з фіч - використання гілок в системі контролю версій. У цьому випадку треба визначити модель розгалуження (наприклад, таку як Gitflow) і описати як код потрапляє у галузі розробки, тестування та продакшену. Для фіч з тривалим циклом розробки створюються окремі фічі-гілки. Після завершення роботи над фічею розробники зливають зміни з фічі-гілки в основну гілку розробки. Такий підхід працює добре, але може бути незручний, якщо одночасно розробляється багато фіч.

Етап складання полягає в автоматизації упаковки необхідного програмного забезпечення, бази даних та інших компонентів. Наприклад, якщо ви розробляєте Java-додаток, то CI запакує всі статичні файли, такі як HTML, CSS та JavaScript, разом із Java-додатком та скриптами бази даних.

CI не тільки запакує всі компоненти програмного забезпечення та бази даних, але й автоматично виконає модульні тести та інші види тестування. Таке тестування дозволяє розробникам отримати зворотний зв'язок у тому, щоб зміни нічого не зламали.

Більшість CI/CD-інструментів дозволяє запускати складання вручну, за комітом або за розкладом. Командам необхідно обговорити розклад збірки, який підходить для них залежно від кількості команди, очікуваної кількості щоденних комітів та інших критеріїв. Важливо, щоб коміти та складання були швидкими, інакше довге складання може стати перешкодою для розробників, які намагаються швидко і часто комітити.

Безперервне тестування – це більше, ніж автоматизація тестування

Фреймворки для автоматизованого тестування допомагають QA-інженерам розробляти, запускати та автоматизувати різні види тестів, які допомагають розробникам відстежувати успішність збирання додатків. Тестування включає функціональні тести, що розробляються в кінці кожного кроку і об'єднуються в регресійні тести для всього додатка. Регресійні тести інформують команду: чи не зламали їх зміни якийсь функціонал в іншій частині програми.

Найкраща практика полягає в тому, щоб вимагати від розробників запускати всі або частину регресійних тестів у локальних оточеннях. Це гарантує, що розробники будуть комітити вже перевірений код.

Регресійні тести – це лише початок. Тестування продуктивності, тестування API, статичний аналіз коду, тестування безпеки – ці та інші види тестування також можна автоматизувати. Ключовим моментом є можливість запуску цих тестів із командного рядка, через веб-хук (webhook) або через веб-сервіс та повернення результату виконання: успішним був тест чи ні.

Безперервне тестування має на увазі не тільки автоматизацію, але і інтеграцію автоматичного тестування в конвеєр CI/CD. Модульні та функціональні тести можуть бути частиною CI та виявляти проблеми до або під час запуску CI-конвеєра. Тести, що вимагають розгортання повного оточення, такі як тестування продуктивності та безпеки, часто є частиною CD та виконуються після розгортання збірок у цільових середовищах.

CD-конвеєр автоматизує доставку змін у різні оточення

Безперервне постачання — це автоматичне розгортання програми в цільове середовище. Зазвичай розробники працюють з одним або декількома середовищами розробки та тестування, в яких програма розгортається для тестування та перегляду. Для цього використовуються такі CI/CD-інструменти як Jenkins, CircleCI, AWS CodeBuild, Azure DevOps, Atlassian Bamboo, Travis CI.

Типовий CD-конвеєр складається з етапів збирання, тестування та розгортання. Більш складні конвеєри включають наступні етапи:

- Отримання коду із системи контролю версій та виконання складання.
- Налаштування інфраструктури, що автоматизується через підхід “інфраструктура як код”.
- Копіювання коду у цільове середовище.
- Налаштування змінних оточення для цільового середовища.
- Розгортання компонентів програми (веб-сервери, API-сервіси, бази даних).
- Виконання додаткових дій, таких як перезапуск сервісів або виклик сервісів, необхідних для працездатності нових змін.
- Виконання тестів та відкат змін оточення у разі провалу тестів.
- Логування та надсилання сповіщень про стан поставки.

Наприклад, у Jenkins конвеєр визначається у файлі Jenkinsfile, в якому описуються різні етапи, такі як складання (build), тестування (test) та розгортання (deploy). Там же описуються змінні оточення, секретні ключі, сертифікати та інші параметри, які можна використовувати на етапах конвеєра. У розділі post налаштовується обробка помилок та сповіщення.

У складнішому CD-конвеєрі можуть бути додаткові етапи, такі як синхронізація даних, архівування інформаційних ресурсів, встановлення оновлень та патчів. CI/CD-інструменти зазвичай підтримують плагіни. Наприклад, у Jenkins є більше 1500 плагінів для інтеграції зі сторонніми платформами, для розширення інтерфейсу користувача, адміністрування, управління вихідним кодом і збіркою.

При використанні CI/CD-інструменту розробники повинні переконатися, що всі параметри налаштовані поза програмою через змінні оточення. CI/CD-інструменти дозволяють встановлювати значення цих змінних, маскувати паролі та ключі облікових записів, а також налаштовувати їх під час розгортання для конкретного оточення.

Також у CD-інструментах присутні дашборди та звітність. У разі збою складання чи постачання вони сповіщають про це. При інтеграції CD із системою контролю версій та agile-інструментами полегшується пошук змін коду та історій користувача, що увійшли до збірки.

Реалізація CI/CD-конвеєрів з Kubernetes та безсерверними архітектурами

Багато команд, що використовують CI/CD-конвеєри у хмарах, використовують контейнери, такі як Docker, та системи оркестрації, такі як Kubernetes. Контейнери дозволяють стандартизувати упаковку, постачання і спростити масштабування та знищення оточення з непостійним навантаженням.

Є безліч варіантів спільного використання контейнерів, інфраструктури як код та CI/CD-конвеєри.

Архітектура безсерверних обчислень є ще одним способом розгортання і масштабування додатків. У безсерверному оточенні інфраструктурою повністю керує постачальник хмарних послуг, а програма споживає ресурси за необхідності відповідно до його налаштувань. Наприклад, в AWS безсерверні програми запускаються через функції AWS Lambda, розгортання яких може бути інтегровано в CI/CD-конвеєр Jenkins за допомогою плагіна.

CI/CD забезпечує розгортання коду

Отже, CI упаковує, тестує збирання та сповіщає розробників, якщо щось пішло не так. CD автоматично розгортає програми та виконує додаткові тести.

CI/CD-конвеєри призначені для організацій, яким необхідно часто вносити зміни до додатків з надійним процесом постачання. Крім стандартизації складання, розробки тестів та автоматизації розгортань ми отримуємо цілісний виробничий процес з розгортання змін коду. Використання CI/CD дозволяє розробникам зосередитися на покращенні програм і не витратити сили на його розгортання.

CI/CD є однією з DevOps-практик, оскільки спрямована на боротьбу з протиріччями між розробниками, які хочуть часто вносити зміни, та експлуатацією, яка потребує стабільності. Завдяки автоматизації, розробники можуть вносити зміни частіше, а команди експлуатації, у свою чергу, набувають більшої стабільності, оскільки конфігурація оточень стандартизована і в процесі постачання здійснюється безперервне тестування. Також налаштування змінних оточення відокремлено від програми та присутні автоматизовані процедури відкату.

Ефект від впровадження CI/CD-конвеєрів можна виміряти у вигляді ключових показників ефективності (KPI) DevOps. Такі KPI як частота постачання (deployment frequency), час реалізації змін (change lead time) та середній час відновлення після інциденту (mean time to recovery) часто покращуються при впровадженні CI/CD з безперервним тестуванням. Однак CI/CD — це лише один із процесів, який може сприяти цим покращенням. Є й інші умови збільшення частоти поставки.

Для початку роботи з CI/CD команді розробників та експлуатації необхідно спільно визначитися з технологіями, практиками та пріоритетами. Команди повинні виробити консенсус щодо правильних підходів до свого бізнесу та технологій, щоб після впровадження CI/CD команда постійно дотримувалася вибраних практик.

Що таке Ansible та як його використовувати

Що таке Ansible? Це програмне забезпечення з відкритим вихідним кодом, яке автоматизує постачання програмного забезпечення, керування конфігурацією та розгортання програм. Ansible допомагає DevOps-фахівцям автоматизувати складні завдання.

Розглянемо питання:

- Ключові особливості програми Ansible
- Встановлення та запуск
- Структура Ansible
- Приклади

Ключові особливості програми Ansible

- **Безагентна.** У клієнті не встановлено програмне забезпечення чи агент, який спілкується із сервером.
- **Ідемпотентна.** Незалежно від того, скільки разів ви виконуєте операцію, результат буде однаковим.
- **Просте та розширюване ПЗ.** Програма Ansible написана на Python та використовує YAML для написання команд. Обидві мови вважаються відносно простими у вивченні.

Встановлення та запуск

ubuntu

```
sudo apt-get install ansible
```

#mac-OS

```
brew install ansible
```

Структура Ansible

Модулі

Це невеликі програми, які виконують певну роботу на сервері. Наприклад, замість запуску цієї команди:

```
sudo apt-get install htop
```

Ми можемо використовувати модуль apt і встановити htop:

```
- name: Install htop
```

```
apt: name=htop
```

Використання модуля дасть вам можливість дізнатися, встановлений він чи ні.

Плагіни

Ansible поставляється з кількома зручними плагінами, і ви можете легко написати свій власний.

Інвентаризація хостів

Щоб надати перелік хостів, нам потрібно позначити список у файлі інвентаризації. Він нагадує зміст файлу hosts.

У найпростішому вигляді він може містити один рядок:

```
127.0.0.1 ansible_ssh_user=ubuntu
```

Playbooks

Ansible playbooks – це спосіб надсилання команд на віддалені комп'ютери за допомогою скриптів. Замість того, щоб індивідуально використовувати команди для віддаленого налаштування комп'ютерів з командного рядка, можна налаштовувати цілі складні середовища, передаючи скрипт однієї або кількох систем.

group_vars

Файл містить набір змінних, наприклад, ім'я користувача та пароль бази даних.

Ролі

Це спосіб згрупувати кілька завдань в один контейнер, щоб ефективно автоматизувати роботу з допомогою зрозумілої структури каталогів.

Оброблювачі

Це списки завдань, які насправді не відрізняються від звичайних завдань, на які посилається глобально унікальне ім'я і які сповіщаються повідомниками. Якщо обробник ні не повідомляє, він не запускатиметься. Незалежно від того, скільки завдань повідомляє обробник, він запускається лише один раз після того, як всі завдання завершені.

Теги

Якщо у вас playbook з великим обсягом, можливо корисно мати можливість запускати тільки певну частину його конфігурації.

Приклади

Рестарт служб PHP та Nginx

```
---
- name: Restart PHP-FPM
  service:
    name: php{{php_version}}-fpm
    state: restarted
  #####
- name: Restart Nginx
  service:
    name: nginx
    state: restarted
```

Встановлення модулів PHP

Щоб викликати обробник, ми повинні використовувати notify: Restart PHP-FPM, імена обробників мають бути унікальними.

```
---
- name: Install PHP {{php_version}} PPA Repo
  apt_repository:
    repo: 'ppa:ondrej/php'
  tags:
    - php
  ##
- name: Install PHP {{php_version}}
  apt: name=php{{php_version}} state=latest
  ##
- name: Install PHP packages
  become: true
  apt:
    name: "{{ item }}"
    state: latest
  with_items:
    - php{{php_version}}-curl
    - php{{php_version}}-fpm
    - php{{php_version}}-intl
    - php{{php_version}}-mysql
    - php{{php_version}}-xml
    - php{{php_version}}-mbstring
  notify: Restart PHP-FPM
  tags:
    - php
```

Тут ми визначили php як тег, тому, наприклад, якщо ви хочете запустити тільки це завдання зі свого playbook, вам необхідно виконати її з `-tags = php`, яка виконуватиме тільки її.

Встановлення Nginx

```
- name: Install Nginx web server
  apt:
    name: nginx
    state: latest
  notify: Restart Nginx
  tags:
    - nginx

###

- name: Update nginx config files
  become: true
  template:
    src: templates/nginx.conf
    dest: "/etc/nginx/sites-available/default"
  tags:
    - nginx
  notify: Restart Nginx

###

- name: link nginx config
  become: true
  file:
    src: "/etc/nginx/sites-available/default"
    dest: "/etc/nginx/sites-enabled/default"
    state: link
  tags:
    - nginx
  notify: Restart Nginx
```

vars.yml

```
ansible_ssh_user: "ubuntu"
current_user: "ubuntu"
server_name: "app_name"

repo_git_url: "app_github_url"
ansible_ssh_private_key_file: "ssh_dir"
php_version: 7.2
app_work_dir: /var/www/app_name/

#mysql config
mysql_host: "mysql_host"
mysql_db: app_name
mysql_user: sql_user
mysql_pass: sql_pass

#other config
cache_driver: file
session_driver: file
app_env: production
app_debug: false
app_key: "your_app_key"
app_name: "app_name"
app_url: "your_app_url"
```

Як використовувати Ansible-Vault

Можна створити секретний файл сховища, який містить ключ шифрування, що шифрує ваші змінні.

```
touch .vault_pass.txt
```

```
echo 'YOUR_CONFIG_PASS' > .vault_pass.txt
```

Щоб зашифрувати змінні, використовуйте:

```
ansible-vault encrypt group_vars/vars.yml --vault-password-file .vault_pass.txt
```

Щоб розшифрувати змінні, використовуйте:

```
ansible-vault decrypt group_vars/vars.yml --vault-password-file .vault_pass.txt
```

Генерування .env

Ansible використовує шаблонизатор Jinja2 для динамічних виразів та доступу до змінних. Створимо файл env.conf.

```
APP_ENV={{app_env}}
APP_DEBUG={{app_debug}}
APP_KEY={{app_key}}
APP_URL={{app_url}}
APP_NAME={{app_name}}

DB_HOST={{mysql_host}}
DB_DATABASE={{mysql_db}}
DB_USERNAME={{mysql_user}}
DB_PASSWORD={{mysql_pass}}

CACHE_DRIVER={{cache_driver}}
SESSION_DRIVER={{session_driver}}
```

Створення playbook

Як видно, ми визначили aws як хост для цього playbook і sudo yes дає нам можливість виконувати команду як користувачу sudo. У нас є vars_files, де ми зберігаємо наші vars. Ми встановили roles, кожна role виконує певну задачу. І, нарешті, у нас є handlers, які містять усі обробники проекту.

Запуск playbook

```
#ansible-playbook playbookName
ansible-playbook code-deploy.yml
```

```
# Запуск з конкретними тегами
ansible-playbook playbook.yml --tags="env-
files,php"
```

```
# Якщо ви використовуєте ansible-vault
ansible-playbook code-deploy.yml --vault-
password-file .vault_pass.txt
```

```
---
- hosts: aws
  #common options between modules
  sudo: yes
  gather_facts: no

  vars_files:
    - ./group_vars/vars.yml
  roles:
    - misc
    - php
    - mysql
    - redis
    - nginx
    - bootstrap-app
    - code-deploy
  ###
  handlers:
    - include: handlers/main.yml
```

Завдання для виконання:

1. Встановити Ansible.
2. Налаштувати плейбук з наступними параметрами:
 - a. Оновлення системи
 - b. Встановлення nginx
 - c. Встановлення VPN сервера
 - d. В плейбуку мають бути теги таким чином, щоб можна було, наприклад, виконати тільки оновлення системи чи рестарт служби nginx.
3. Оформити звіт про виконання роботи зі знімками екрана (або його частини), які ілюструють виконання завдань.